

lsmeear: selección de variables con multiplicadores de Lagrange

J Glob Optim (2017)

Ignacio Araya Bertrand Neveu

La selección de variables domina el costo

- El solver resuelve problemas globales continuos de forma determinista.
- Cada bisección divide el box actual en dos subproblemas.
- La elección de x_i afecta poda y profundidad del árbol.
- Una mala regla puede disparar el número de nodos.
- El objetivo es una estrategia **informada y barata**.

Las reglas clásicas tratan todas las restricciones igual

- `rr` ignora la estructura del sistema; `lf` solo mira el ancho del dominio.
- `smax`, `ssum` y `ssr` agregan `smear` sobre todas las restricciones.
- Todas pesan por igual restricciones activas e inactivas.
- Esa uniformidad limita su capacidad de priorizar impacto.

ViolationTransfer guía la elección, pero es intrusivo

- Usa multiplicadores duales de una relajación convexa.
- Pondera la importancia con la función lagrangiana relajada.
- Requiere reformular el problema y modificar el contractor.
- La variable se fija antes de rebisecar el nodo.
- Es más complejo que una linealización simple.

lsmeas pondera las restricciones antes de medir smear

- Linealiza cada restricción alrededor del punto medio del box actual.
- Resuelve un LP para obtener los multiplicadores duales óptimos λ^* .
- Construye una **Lagrangiana** con esos pesos y mide el impacto de cada variable.
- Usa el Jacobiano intervalar para estimar el efecto de x sobre L .
- Así favorece las restricciones más relevantes al decidir qué variable bisecar.

lsmeasr elige la variable con mayor impacto

- lsmeasr linealiza con el término de primer orden de Taylor en el punto medio de x .
- El LP añade cotas; si falla o es no acotado, cae en `smearsum`.
- **Impacto** de x_i : $|D_i \text{wid}(x_i)|$, con D_i usando λ^* y J .
- Para cada variable, se acumulan derivadas intervalares de restricciones ponderadas por multiplicadores duales óptimos.
- Se bisecciona la variable con impacto máximo y la implementación base no altera el contractor.

Las pruebas usan IbexOpt y COCONUT

- Implementamos `lsmeas` y sus variantes en `IbexOpt`, sobre `Ibex`.
- `IbexOpt` usa `HC4`, `ACID(HC4)`, `AF2`, `XNewton`, `inHC4` e `InnerPolytope`.
- La selección del siguiente nodo se hace con `FeasibleDiving`.
- Se evaluaron 76 instancias de las series 1 y 2 de COCONUT.
- Se fijó $\epsilon = 10^{-6}$, con cinco corridas por instancia y promedio robusto.
- Se comparó el desempeño en tiempo de CPU y tamaño del árbol de búsqueda.

lsmeat supera a las reglas clásicas

- Se comparó lsmeat con rr, lf, smax, ssum y ssr.
- lsmeat domina en tiempo y nodos; lf da los peores resultados.
- Casi 90% de instancias quedan bajo dos veces el mejor tiempo.
- El mejor promedio relativo frente a ssum es 1.30.
- lsmeat logra el mejor equilibrio entre tiempo y nodos.
- El LP añade sobrecarga pequeña frente al filtrado, que ya resuelve hasta $2n$ LPs.

lsmear-MG y lsmear-LR lideran las variantes

- lsmear-MG calcula la linealización con el gradiente en el punto medio de la caja.
- lsmear-LI y lsmear-LRI usan la relajación lineal para ponderar el impacto.
- lsmear-MG obtiene el mejor promedio global y lsmear-LR queda muy cerca.
- lsmear-LI y lsmear-LRI quedan rezagadas frente a las otras variantes.
- La variante simple lsmear-MG gana sin modificar el contractor del solver.

La búsqueda por FeasibleDiving potencia el método

- FeasibleDiving supera a minLB con cualquier selector de variables.
- lsmeas-MG sigue siendo el mejor bajo ambas búsquedas.
- Con minLB, lsmeas-MG mejora a ssum por 1.63.
- Con FeasibleDiving, la mejora se mantiene en 1.43.
- La elección de nodos y variables se refuerzan mutuamente.

La idea central es ponderar, no solo medir

- Las restricciones activas merecen más peso que las inactivas.
- La linealización da pesos baratos y generalizables.
- `lsmer-MG` conserva la simplicidad de implementación.
- La ganancia viene de priorizar información relevante.
- Evita depender de una relajación convexa compleja.

El siguiente paso es incorporar violación de restricciones

- Integrar el criterio de violación de `ViolationTransfer` en la selección de variables.
- Ignorar variables ligadas **solo** a restricciones lineales, porque ya se reducen bien.
- Mantener el efecto indirecto sobre variables no lineales al explorar particiones alternativas.
- Afinar la combinación entre pesos, violación e impacto para priorizar mejor cada variable.
- Extender esta idea a otros `solvers` intervalares, preservando independencia del contractor.